

DeepShare: Assurance-Driven Deep Learning Job Scheduling for Multi-Tenant Clusters

Anonymous Authors

Abstract—Multi-tenant GPU clusters frequently remain underutilized even when tenants experience long queueing delays, because quota control, queue ordering, preemption, and GPU sharing are driven by different local signals. We present DeepShare, a scheduler that uses a continuous tenant-assurance signal to coordinate these decisions at runtime. DeepShare combines elastic quota borrowing, tenant-specific runtime prediction, cost-aware best-effort preemption, and interference-aware MPS colocation, while using the same assurance signal to decide when borrowed capacity should be reclaimed and when sharing should become more conservative. In trace-driven experiments on 23,859 Venus jobs and 3,200 internal jobs, DeepShare achieves an average GPU utilization of 70.58%, a 29.5% improvement over the strongest non-intrusive sharing baseline, while reducing average queueing delay by 46%. On a 16-GPU Kubernetes testbed, it reduces the average job completion time by 34% and maintains 93% QoS compliance for guaranteed tenants. These results show that treating tenant assurance as a runtime control loop achieves a more advantageous utilization-QoS trade-off than optimizing quotas, scheduling, and resource sharing independently.

I. INTRODUCTION

Deep learning (DL) training workloads [1], [2], [3] have become primary consumers of GPU resources in modern data centers, powering applications in computer vision [4], mathematical reasoning [5], and scientific research [6]. To improve hardware efficiency, industry and academia widely adopt multi-tenant GPU clusters [7], [8], where resources are centrally managed across users and teams. In practice, such clusters often host long-running training, short exploratory jobs, and opportunistic experiments, making static resource allocation difficult to keep efficient. In such clusters, unused capacity cannot simply be reassigned without affecting tenant guarantees, yet production deployments still leave large GPU fractions idle or lightly used [9], [10]. On Alibaba’s PAI platform, more than 75% of tasks request fewer than 10% of a GPU, while average utilization remains only 25–50% [10], [11]; similar inefficiency appears in large-scale clusters [12], where compute and memory utilization remain low despite high demand.

This underutilization stems not only from static allocation but also from resource-management decisions made with little awareness of one another. Quota mechanisms assign tenant entitlements, queueing policies choose the next job, and sharing policies decide whether two jobs can safely share a GPU. The question is not only how to find idle GPUs, but also when their use remains safe for future quota recovery. These choices interact: borrowed quota must be reclaimable, short-job priority must not delay under-served tenants, and low-

interference colocation may still be undesirable if it slows quota recovery [9], [10], [13].

Recent work has improved individual parts of GPU cluster management, including job colocation [14], [15], performance-aware scheduling [16], [17], [18], and fairness-oriented allocation [19]. Yet framework-level sharing mechanisms [8], [20] often sacrifice portability, fragmentation-aware strategies rely on static boundaries, runtime prediction approaches [21], [22] are sensitive to workload regularity, and fairness models provide limited support for per-tenant service differentiation. A locally attractive decision, such as admitting a short job or a low-interference pair, can still be harmful if it delays quota recovery. The missing link is runtime coordination: a scheduler must know not only whether a job is short or a pair has low interference, but also whether admitting it helps or hurts recovery of tenants whose guarantees are unmet.

We introduce DeepShare, a Kubernetes-native resource management framework built around the *Quota Assurance Degree* $\tilde{Q}_i(t)$. QAD measures how far tenant i is from receiving its guaranteed share over time: high QAD means guaranteed demand is served, while low QAD exposes under-service and raises recovery priority. By comparing allocation with the smaller of quota and current guaranteed demand, QAD avoids rewarding artificial demand inflation or penalizing temporarily idle quota. Smoothing avoids reacting to short-lived fluctuations while preserving persistent deficits for the scheduler. Best-effort¹ resources are reclaimed when they interfere with recovery.

DeepShare uses QAD across scheduling decisions. Deficit-aware Reclaimable Allocation (DRA, §III-A) exposes idle capacity to best-effort workloads, while QAD decides when borrowed capacity gives way to guaranteed demand. Runtime prediction (§III-B) is applied after tenant recovery priority, so short jobs are favored only among tenants with comparable assurance states. Interference-aware colocation (§III-C) considers predicted slowdown and tenant assurance, becoming more conservative when either tenant is under-served, without modifying user code or DL frameworks. As a tenant recovers, the same signal relaxes these decisions and re-enables more aggressive sharing.

Evaluation shows that DeepShare increases average GPU utilization to 70.58% in trace-driven simulations, a 29.5% improvement over Lucid [23], while reducing average queueing delay by 46%. On a Kubernetes testbed, DeepShare reduces

¹In this paper, best-effort jobs consume reclaimable surplus capacity beyond a tenant’s quota and may be preempted when guaranteed demand rises.

average job completion time by 34%, queueing delay by 66%, and maintains 93% QoS compliance for guaranteed tenants [24], [25]. Ablation shows that elastic borrowing, runtime-aware ordering, and QAD-aware colocation reduce queueing delay by 31% beyond DRA (§III-A) alone.

The main contributions of this paper are:

- QAD, a continuous tenant-assurance measure that compares recovery urgency across tenants and distinguishes transient quota fluctuations from persistent under-service.
- An assurance-aware scheduling policy that applies QAD before runtime prediction, reducing queueing delay without letting short-job optimization override tenant recovery.
- A Kubernetes-native scheduler with reclaimable best-effort borrowing, cost-sensitive preemption, and QAD-aware MPS colocation, evaluated through trace-driven simulation and a 16-GPU deployment.

II. BACKGROUND AND MOTIVATION

A. GPU Cluster Scheduling and Sharing

Multi-tenant GPU clusters have become the primary platform for deep learning (DL) research and deployment [9], [10], [26], [27]. For isolation and fairness, they are commonly partitioned into *virtual clusters* (VCs) with fixed resource quotas [14], [28]. Fixed quotas make resource allocation predictable, but they also make it difficult to reuse temporarily idle capacity across tenants.

At the same time, advances in GPU compute capability have increased the gap between hardware capacity and the resource demands of many individual DL jobs. GPU sharing [20] addresses this gap at multiple levels—hardware partitioning (e.g., NVIDIA MIG [29]), compute-level multiplexing (e.g., MPS [30]), and driver-level software virtualization—each trading isolation against flexibility. A growing body of schedulers builds on these mechanisms to colocate multiple jobs per GPU [8], [15], [20], [23], [31], [32], employing time multiplexing, spatial partitioning, or interference-aware placement combined with higher-level scheduling policies [20], [23].

B. Empirical Study of a Multi-Tenant GPU Cluster

Although administratively convenient, fixed quotas introduce a temporal mismatch: they are provisioned on weekly or monthly timescales, while DL workloads are bursty over hours or minutes. This misalignment leads to jobs queueing behind quota limits even as GPUs remain underutilized, in Figure 1. Production studies report overall GPU utilization of only 25%–52% [9], [10], [11].

To inform our design, we profiled a university-managed GPU cluster with 58 nodes and 219 GPUs, including NVIDIA Tesla V100-32GB and A100-40GB devices, partitioned into 12 virtual clusters with fixed monthly quotas. We collected one week of telemetry via NVIDIA DCGM, including GPU-level metrics (e.g., SM utilization, memory bandwidth), job-level statistics (e.g., resource requests), and user-level submission patterns.

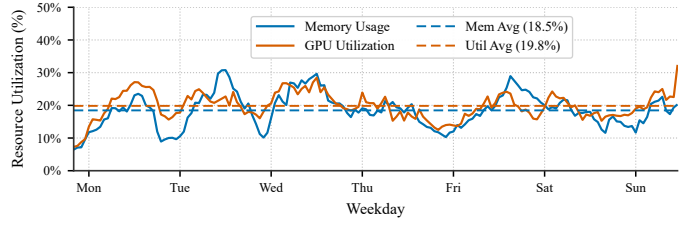


Fig. 1: GPU and memory utilization across one week.

Observation 1: Persistent Underutilization. Figure 1 highlights a systematic mismatch between queueing pressure and actual GPU usage. During the seven-day period, average GPU compute utilization is 19.8% and average memory utilization is 18.5%. A clear diurnal rhythm emerges: arrivals are concentrated during daytime hours (e.g., 10:00–12:00, 14:00–18:00), with substantially lower submission rates at night. Burst submissions fully consume group quotas, after which allocations remain idle until the next active window.

Observation 2: Heterogeneity and Colocation Potential. The workload shows strong heterogeneity along two dimensions. In terms of duration, short exploratory jobs (median runtime under 30 minutes) constitute 62% of submissions but use only 11% of GPU-hours, whereas long training jobs (over 4 hours) consume 73% of GPU-hours. In terms of resource profile, 41% of simultaneously running job pairs have complementary behavior: one is compute-heavy (SM utilization > 50%) and memory-light (< 40% device memory), while the other is memory-heavy or compute-light. Even under a conservative 80% combined-utilization cap, many concurrent pairs could safely share a GPU under the current exclusive-assignment policy.

Observation 3: Short-Running Job Starvation. Under the existing FIFO scheduler, short jobs (62% of all submissions) incur a median queueing delay of 47 minutes, which even exceeds their own median runtime of less than 30 minutes. This unfairness occurs because the scheduler assumes runtimes are unknown and thus cannot give precedence to short jobs over long ones. However, user submission behavior is structured enough to allow data-driven prediction: per-user runtime distributions exhibit low variability, and 78% of users repeatedly submit jobs with similar characteristics and durations. Reliable runtime estimates make it possible to schedule short jobs first, avoid preempting nearly completed jobs, and predict the lifetime of colocated job pairs.

C. Motivation

The observations above suggest that underutilization arises primarily from rigid resource-management policies rather than insufficient workload demand. Improving utilization therefore requires elastic borrowing across tenants, differentiated treatment of guaranteed and opportunistic work, runtime-aware scheduling to reduce head-of-line blocking, and interference-aware colocation to exploit complementary jobs. These mechanisms must also be tied to tenant assurance: borrowed

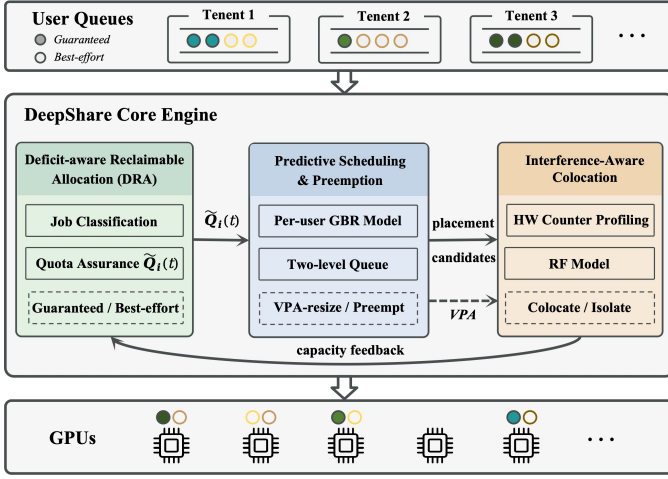


Fig. 2: The architecture of DeepShare

TABLE I: Notation and definitions.

Symbol	Description
K, G_{tot}	Number of GPUs and total GPU capacity
G_f, G_p	Free GPUs and total pending GPU demand
J_k	Jobs resident on GPU k
R_j	GPU demand of job j
g	GPU demand of an incoming Guaranteed job
i, q_i	Tenant index and Guaranteed GPU quota
A_i^G, D_i^G	Allocated and demanded Guaranteed GPUs
U_i^G, U_i^B	Current Guaranteed and Best-effort GPU usage
$Q_i, \tilde{Q}_i$	Instantaneous and EMA-smoothed QAD
η, τ_p	Best-effort cap multiplier and preemption window
Q_i^G, Q_i^B	Tenant-level Guaranteed and Best-effort queues
$\tilde{Q}^G, \tilde{Q}^B$	Cluster-level Guaranteed and Best-effort queues
$\hat{T}(j), \bar{T}(J)$	Predicted remaining runtime and victim-set mean runtime
$C_p(j), \Phi(J), \varepsilon_j$	Normalized preemption overhead, victim-set cost, and preemption efficiency
$\rho, \hat{\rho}$	Retention ratio and predicted retention
$\rho_{\text{tol}}, \rho_{\text{min}}$	Admission tolerance and retention floor
P	Cluster contention pressure

resources should remain reclaimable, short-job acceleration should not postpone an under-served tenant, and colocation should become more conservative when it may slow quota recovery. At the same time, practical deployment demands framework-agnostic designs to avoid per-framework maintenance overhead.

III. SYSTEM OVERVIEW

DeepShare organizes scheduling decisions around a tenant-assurance state, the Quota Assurance Degree $\tilde{Q}_i(t)$, as shown in Figure 2. DeepShare uses this state in three places: DRA (§III-A) decides when idle quota can be borrowed or reclaimed, predictive scheduling and preemption (§III-B) order jobs and select low-cost victims, and interference-aware colocation (§III-C) decides when two jobs can safely share a GPU without delaying tenant recovery.

A. DRA: Deficit-aware Reclaimable Allocation

Strictly static quota enforcement causes GPUs to be underutilized when multi-tenant demand is bursty, even though

DL workloads differ in how much reclaimable excess capacity they can safely tolerate, analogous to oversubscription [12]. DeepShare introduces *Deficit-aware Reclaimable Allocation* (DRA), which addresses these issues by translating quota fulfillment into a continuous control signal and separating scheduling semantics from the underlying physical placement.

Workload classes. DeepShare targets DL training workloads and distinguishes between quota-backed and opportunistic execution [7], [33]. Each tenant labels submitted jobs as either *Guaranteed* or *Best-effort*. *Guaranteed* jobs are charged against the quota q_i and prioritized for placement backed by the quota. *Best-effort* jobs are not charged against the quota and incur negligible scheduling cost; they utilize surplus capacity and remain reclaimable when *Guaranteed* demand is underserved.

Quota assurance degree. For each tenant i , DeepShare defines the instantaneous Quota Assurance Degree (QAD) as in Equation 1. QAD contrasts the tenant’s current guaranteed allocation with $\min(q_i, D_i^G(t))$, where $D_i^G(t)$ is its present guaranteed demand. This definition prevents tenants from boosting their priority by declaring demand far above their quota, while also ensuring they are not penalized for leaving part of their quota unused. In this way, QAD measures how effectively the scheduler fulfills the guaranteed demand that the tenant is entitled to receive.

$$Q_i(t) = \begin{cases} 1, & D_i^G(t) = 0, \\ A_i^G(t)/\min(q_i, D_i^G(t)), & D_i^G(t) > 0. \end{cases} \quad (1)$$

Temporal smoothing and control. Binary quota mechanisms (e.g., Kubernetes ElasticQuota and Volcano [24], [25]) provide only a coarse indication of whether a tenant is within its quota. This granularity is insufficient for prioritizing recovery among under-served tenants or regulating the aggressiveness of sharing. In contrast, QAD provides a continuous signal: lower values indicate greater deficit, while smoothing mitigates transient fluctuations due to short job completions or bursty arrivals. To stabilize this signal, DeepShare applies exponential moving average (EMA) smoothing (Equation 2) with default $\lambda = 0.3$. Under a 50ms scheduling cycle, a sustained deficit contributes 90% of its steady-state effect within ~ 350 ms, enabling timely reclamation while filtering sub-second noise.

The smoothed QAD, $\tilde{Q}_i(t)$, drives multiple control decisions. *Guaranteed* jobs are ordered by $(\tilde{Q}_i(t)! \uparrow, \hat{T}(j)! \uparrow)$; colocation admission is tightened when either tenant is under-served; and reclamation is prioritized before considering *Best-effort* preemption. *Best-effort* allocations do not contribute to $A_i^G(t)$ and therefore do not inflate QAD. Preemption is triggered by placement failure.

$$\tilde{Q}_i(t) = \lambda Q_i(t) + (1 - \lambda) \tilde{Q}_i(t - 1) \quad (2)$$

Recovery dynamics. When *Guaranteed* demand is feasible, i.e., $\sum_i \min(q_i, D_i^G(t)) \leq G_{\text{tot}}$, a *Guaranteed* job blocked

solely by *Best-effort* occupancy is admitted within one preemption window τ_p , subject to Kubernetes Pod termination latency and a bounded *Best-effort* cleanup delay δ . Once $A_i^G = \min(q_i, D_i^G)$, we have $Q_i = 1$, and the EMA converges geometrically at rate $(1 - \lambda)$ per scheduling cycle.

Under overload, where $\sum_i \min(q_i, D_i^G(t)) > G_{\text{tot}}$, full recovery is infeasible. In this regime, prioritizing tenants in ascending order of $\tilde{Q}_i$ approximates max-min fair recovery among active tenants.

B. Predictive Scheduling and Preemption

Conventional schedulers [7], [33] generally assume unknown training durations and use FIFO or static priorities, which causes head-of-line blocking. While integrating runtime prediction can alleviate this, prioritizing jobs solely by predicted time can undermine tenant-level guarantees.

DeepShare instead adopts a lexicographic policy: *Guaranteed* jobs are ordered first by QAD and then by the predicted remaining time. QAD determines inter-tenant recovery priority, while prediction refines intra-tenant ordering among jobs with similar assurance levels. This design improves responsiveness and reduces unnecessary preemption of *Best-effort* jobs that are near completion.

Hierarchical queueing. DeepShare implements tenant-level queues for admission control and cluster-level queues for placement. A *Guaranteed* job is admitted when $U_i^G + R_j \leq q_i$. A *Best-effort* job is admitted only if no pending *Guaranteed* job is placeable via exclusive allocation or safe colocation, and if $U_i^B + R_j \leq \eta q_i$, with default $\eta = 2$. Here, U_i^G and U_i^B denote the tenant's current GPU usage for *Guaranteed* and *Best-effort* jobs, respectively. Admission does not consume the quota until placement succeeds.

At the cluster level, jobs are scheduled using a lexicographic priority. *Guaranteed* jobs are considered before *Best-effort* jobs; within each class, jobs are ordered by $(\tilde{Q}_i(t)! \uparrow, \hat{T}(j)! \uparrow)$. Because QAD is the primary key, prediction error affects only local ordering among similarly served tenants and cannot elevate a well-served tenant above an under-served one.

Cost-based preemption. Unlike progress-agnostic policies, DeepShare performs cost-aware victim selection that explicitly accounts for both job progress and preemption history. Given a target demand of g GPUs, the scheduler selects a victim set J such that the total number of fully released GPUs satisfies via Equation 3. This formulation naturally handles both multi-GPU jobs and colocated workloads: a shared GPU is considered available only when all resident jobs on that device are preempted.

$$\sum_{k=1}^K \mathbb{I}[\mathcal{J}_k \neq \emptyset \wedge \mathcal{J}_k \subseteq J] \geq g. \quad (3)$$

DeepShare also estimates each job's remaining runtime $\hat{T}(j)$ using a per-tenant gradient boosting regressor [34]. We define the preemption cost of a victim set J as Equation 4, where $\bar{T}(J) = |J|^{-1} \sum_{j \in J} \hat{T}(j)$ denotes the mean remaining runtime, and $C_p(j)$ captures the normalized cumulative

Algorithm 1: Scheduling Strategy

Input : Tenants U , cluster state
Output: Updated placements

```

1 foreach tenant  $i \in U$  do
2   foreach job  $j \in Q_i^G$  do
3     if  $U_i^G + R_j \leq q_i$  then
4        $\text{Move } j \text{ to } Q^G$ 
5 Sort  $Q^G$  by  $(\tilde{Q}_i(t) \uparrow, \hat{T}(j) \uparrow)$ 
6 PlaceJobs( $Q^G$ , true)
7 if no remaining  $j \in Q^G$  is placeable then
8   foreach tenant  $i \in U$  do
9     foreach job  $j \in Q_i^B$  do
10      if  $U_i^B + R_j \leq \eta q_i$  then
11         $\text{Move } j \text{ to } Q^B$ 
12 Sort  $Q^B$  by  $(\tilde{Q}_i(t) \uparrow, \hat{T}(j) \uparrow)$ 
13 PlaceJobs( $Q^B$ , false)

```

preemption overhead of job j . The first term prioritizes preempting long-running jobs with limited prior disruption, while the second term penalizes fragmented victim sets, particularly those consisting of near-completion jobs (i.e., small $\bar{T}(J)$). We set $\alpha = 0.5$ and $\beta = 0.3$ by default.

$$\Phi(J) = \sum_{j \in J} \frac{1 + \alpha C_p(j)}{\hat{T}(j)} + \frac{\beta(|J| - 1)}{\bar{T}(J)}, \quad (4)$$

For efficient online selection, DeepShare adopts a greedy approximation based on per-job preemption efficiency via $\mathcal{E}_j = \frac{R_j \hat{T}(j)}{1 + \alpha C_p(j)}$, which quantifies the GPU-time reclaimed per unit disruption. The SelectVictims procedure ranks candidate jobs by $\mathcal{E}_j$ and iteratively admits them until the constraint in Equation 3 is satisfied, yielding $O(n \log n)$ complexity for n preemptable *Best-effort* jobs.

In summary, Algorithm 2 considers placement options in order of increasing disruption: exclusive allocation, interference-aware colocation, CPU/memory reclamation, and finally GPU preemption.

C. Interference-Aware Job Colocation

Sharing GPUs can increase utilization, but a low-interference pairing is not always a good choice. If one tenant is currently running below its guaranteed allocation, even a modest slowdown can postpone its recovery. DeepShare therefore treats colocation as an admission-control decision: a pair is scheduled together only when the predicted throughput retention is high enough given both the current cluster load and the assurance states of the tenants involved.

Performance retention prediction. DeepShare estimates pairwise interference via the throughput retention ratio $\rho = t_{\text{shared}}/t_{\text{excl}}$, where t_{shared} and t_{excl} denote the training throughput in colocated and exclusive execution, respectively.

Algorithm 2: Interference-Aware Placement

Input : Queue Q_{in} , flag *allowPreempt*, cluster state
Output: Updated job-to-GPU assignments

```

1 foreach job  $j \in Q_{in}$  in priority order do
2   if  $\exists$  free device with GPU, CPU,
   and memory headroom for  $j$  then
3     Assign  $j$  to  $d$  exclusively
4     continue
5    $P \leftarrow (G_p / (G_f + \epsilon))^\gamma$ 
6    $\mathcal{C} \leftarrow \{(d, j_e) \mid d \text{ hosts exactly one job } j_e\}$ 
7   Sort  $\mathcal{C}$  by  $(\min(\hat{\rho}(j), \hat{\rho}(j_e)) \downarrow, \hat{T}(j_e) \uparrow)$ 
8   foreach  $(d, j_e) \in \mathcal{C}$  do
9      $i \leftarrow \text{owner}(j); i_e \leftarrow \text{owner}(j_e)$ 
10     $\rho_{tol} \leftarrow \min(1, [\rho_{min} + P(1 - \rho_{min})] \max(k - \tilde{Q}_i(t), k - \tilde{Q}_{i_e}(t)))$ 
11    if ( $j$  or  $j_e$  is BE) and  $\hat{\rho}(j) \geq \rho_{tol}$  and
       $\hat{\rho}(j_e) \geq \rho_{tol}$  then
12      Colocate  $j$  on  $d$ 
13      continue
14  if allowPreempt then
15    Shrink BE Pods via in-place resize
16    Update per-node CPU/mem headroom
17    if some node now admits  $j$  on a free
      or colocatable GPU then
18      Retry exclusive or colocation placement
19      continue
20   $J^* \leftarrow \text{SelectVictims}(R_j)$ 
21  Preempt  $J^*$  and assign  $j$  after release

```

The predictor leverages DCGM hardware telemetry, including SM activity, memory bandwidth, L2 cache behavior, PCIe traffic, tensor core utilization, DRAM throughput, and power draw, to capture both compute and memory contention effects. We employ a Random Forest model due to its favorable trade-off between prediction accuracy and sub-millisecond inference latency (§V), which allows online deployment on the critical path of the scheduler. Feature selection via Recursive Feature Elimination indicates that SM activity and memory bandwidth are the dominant predictors of interference, consistent with previous observations on GPU contention (Figure 3).

Dynamic tolerance. Let G_p be total pending GPU demand and G_f the number of free GPUs. Cluster contention pressure is $P = \left(\frac{G_p}{G_f + \epsilon}\right)^\gamma$, where ϵ avoids division by zero. We set $\gamma = 0.5$ so pressure grows sub-linearly, preventing short demand bursts from disabling all colocation. For incoming job j from tenant i and resident job j_e from tenant i_e , the admission tolerance is defined in Equation 5, where k denotes the maximum number of jobs colocated on a single GPU (default $k = 2$).

$$\rho_{tol} = \min(1, [\rho_{min} + P(1 - \rho_{min})] \max(k - \tilde{Q}_i(t), k - \tilde{Q}_{i_e}(t))). \quad (5)$$

A pair is admitted only if it fits GPU memory and node-level CPU/memory headroom, at least one partner is *Best-effort*, and both $\hat{\rho}(j)$ and $\hat{\rho}(j_e)$ exceed ρ_{tol} . The $[\rho_{min} + P(1 - \rho_{min})]$ term raises the tolerance when pending demand is high. The clamp at one is deliberate: when contention is high or one of the tenants is significantly under-served, the system stops creating new colocations and the scheduler focuses on restoring exclusive capacity first.

Thus, admission behaves conservatively when the cluster is under heavy load or when a tenant’s recovery is urgent, and only becomes more permissive when the overall pressure is low and both tenants are considered sufficiently protected.

Runtime validation and overhead. The predictor filters pairs before GPU sharing is enabled, but interference can change across training phases. DCGM counters feed an online retention estimator. If either job’s observed retention stays below ρ_{tol} for w consecutive samples, the pair is marked degraded and the *Best-effort* job is preempted in the next cycle. The fixed-window rule provides bounded detection delay with one counter per pair, cheaper and more predictable than sequential detectors such as CUSUM.

Let m be the number of GPUs hosting exactly one resident job. Candidate generation and prediction are $O(m)$ after pruning, and sorting costs $O(m \log m)$. In practice, the scheduler’s node-scoring phase (§IV-A) caps the evaluated candidates per job, keeping colocation admission within the scheduling latency budget reported in §IV-C.

IV. SYSTEM IMPLEMENTATION

The prototype contains roughly 11.3K lines of code: Go for the scheduler plugin, quota controller, and node-local DaemonSets; Python for the Random Forest based prediction service and per-tenant runtime estimator; and Helm/RBAC manifests plus lightweight quota CRDs for deployment. All mechanisms run on the cluster-management layer and do not require changes to user code or DL frameworks.

A. Scheduler Critical Path

DeepShare is packaged as an out-of-tree scheduler plugin that registers with the standard Kubernetes scheduling framework [24]. Tenant quotas and Best-effort cap multipliers are configured through a lightweight TenantQuota CRD, while individual jobs specify their service class using the optional annotation `deepshare.io/class: guaranteed|best-effort`.

A lightweight quota controller reconciles TenantQuota objects and job-class annotations into per-tenant quota metadata, but it does not make placement decisions. The scheduler plugin keeps the real-time control loop: it maintains $Q_i(t)$ and $\tilde{Q}_i(t)$ in memory, orders queues, admits colocation pairs, and invokes preemption. The QAD state is derivable from running Pods in the informer cache; after failover, the newly elected leader reconstructs it and warm-starts the EMA with the first cycle’s instantaneous QAD, avoiding extra writes or storage.

The plugin maps DeepShare’s policy to five extension points. (i) *Filter* keeps nodes that can host the incoming

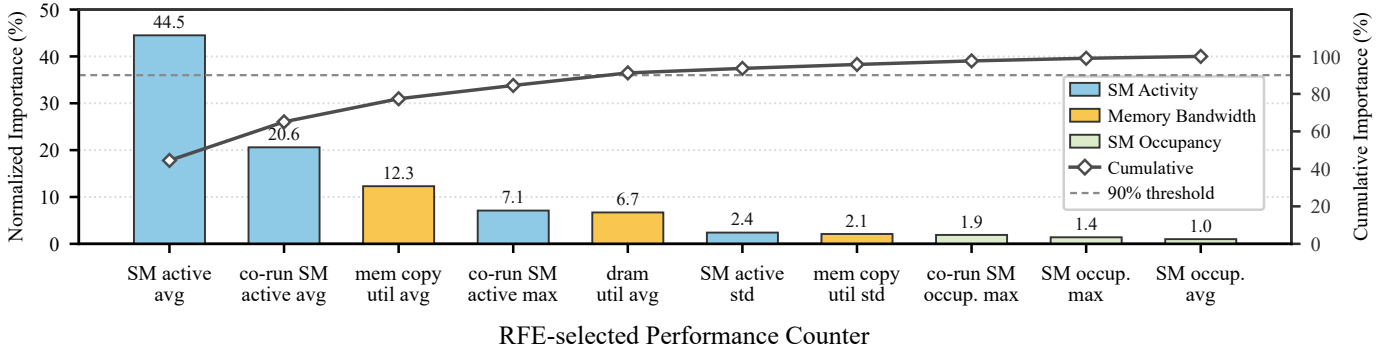


Fig. 3: Feature importance scores via Recursive Feature Elimination with the Random Forest Regressor.

Pod either on a free GPU or on an eligible single-resident GPU under CPU, memory, GPU-memory, and Best-effort cap constraints. (ii) *Score* ranks these candidates using the Random Forest interference model and the bilateral tolerance in §III-C. (iii) *Reserve* records GPU claims in the next-cycle cluster view to avoid double booking. (iv) *PostFilter* runs the cost-based preemption search (§III-B) when no feasible GPU remains. (v) *Permit* holds an incoming Pod only when the VPA-based CPU/memory reclamation step in Algorithm 2 has issued an in-place resize and waits until `Pod.Status.Resources` reflects the reduced allocation.

B. GPU Sharing and Runtime Protection

Spatial sharing is provided by NVIDIA MPS. One MPS control daemon is deployed per GPU in a node-local DaemonSet; it brokers client connections and sets per-client memory limits through MPS controls where supported. Because MPS multiplexes clients rather than isolating SM or memory-bandwidth contention, DeepShare treats sharing as an admission-control problem governed by the interference model in §III-C.

A DCGM poller samples each colocated pair. If observed retention stays below the bilateral ρ_{tol} for three consecutive windows, the poller raises a degradation event; the scheduler consumes it in the next cycle and preempts the Best-effort partner. A DaemonSet recovery hook clears stale MPS client contexts after abnormal Pod exits.

In-place resize. Because `nvidia.com/gpu` is defined as a Kubernetes Extended Resource, its allocation cannot be modified once the Pod has been admitted. The scheduler relies on the Pod `resize` subresource, which is enabled via `InPlacePodVerticalScaling` on our control plane, and it maintains CPU and memory headroom above the VPA recommendations of `max(10%, 0.5 core)` and `max(10%, 256 MB)`, respectively.

C. Scheduling Latency and Fault Tolerance

End-to-end scheduling latency stays below 50 ms per job, dominated by the Kubernetes bind round trip. Feature extraction, queue bookkeeping, and capped interference scoring together account for less than 25 ms, keeping the latency envelope on par with Lucid [23].



Fig. 4: Predicted vs. actual job execution times for 2,000 randomly sampled jobs from the Venus dataset.

The plugin runs as a replicated Deployment with leader election through `coordination.k8s.io/Lease`. With Kubernetes default lease settings, a newly elected replica begins scheduling within one renewal period. Because preemption is expressed as Pod deletion and reconciled idempotently by the API server, no bespoke compensation logic is needed if leadership changes mid-cycle.

V. EXPERIMENTS

This section assesses DeepShare in terms of prediction accuracy, colocation efficiency, multi-tenant quota management, and deployment in a real cluster.

A. Experiment Setup

Platform. We evaluate DeepShare on a Kubernetes-managed GPU cluster with 58 nodes and 219 GPUs, including NVIDIA Tesla V100-SXM2-32GB and NVIDIA A100-PCIE-40GB devices. GPU nodes contain heterogeneous accelerator configurations with 4–8 GPUs per node, high-core-count CPUs, and 256–512 GB memory. The software stack includes Ubuntu 22.04, Kubernetes v1.35 with `InPlacePodVerticalScaling`, CUDA 12.8, and NVIDIA DCGM 3.3.8. The master node runs the scheduler and monitoring services, while GPU nodes host the MPS DaemonSet and training workloads.

Workloads. We used two real-world datasets, summarized in Table II. The Venus trace [26] contains large-scale GPU datacenter scheduling and utilization records from September 2020, providing diverse public workloads for reproducible

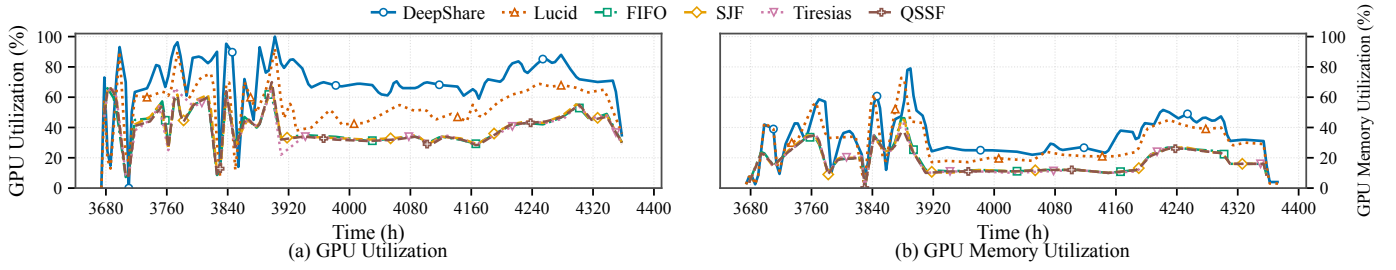


Fig. 5: GPU and GPU memory utilization over time.

TABLE II: Characteristics of the simulation datasets.

Dataset	Source	Job Count	Avg. Execution Time
Venus [26]	Public dataset	23,859	5,419 s
Internal	Internal cluster	3,200	36,887 s

evaluation. The internal trace preserves user quotas, submissions, and allocation policies from our institutional cluster; its longer average runtime (36,887 s vs. 5,419 s) stresses scheduling under long-lived occupancy.

Trace replay methodology. We implemented a trace-driven simulator that reproduces DeepShare’s two-level queues, DRA module, preemption, and interference-aware colocation. It supports FIFO, SJF, QSSF, Tiresias, Lucid, and DeepShare’s strategies.

For the Venus dataset, which does not include per-tenant quota assignments, we synthesize a multi-tenant quota configuration as follows: jobs are partitioned into 12 virtual clusters by user ID hash, and each virtual cluster receives a fixed quota proportional to its historical GPUs-hour share (range: 4-32 GPUs), mirroring the quota distribution observed in our university cluster (§II-B).

Metrics. We evaluated the performance of our scheduling strategies using the following metrics: (1) *Job Completion Time* (JCT): time from job submission to completion, encompassing both queueing and execution time; (2) *Average Queueing Delay*: average time a job spends in the queue before being scheduled; (3) *Makespan*: total time from the first submission to the last completion; (4) *Cluster GPU Utilization*: average percentage of GPU compute resources used; (5) *GPU Memory Utilization*: average percentage of GPU memory used; and (6) *Quota Assurance Degree*: as defined in Eq. 1, measuring SLA fulfilment.

Baselines. We compare DeepShare against five baselines: FIFO, SJF, QSSF [26], Tiresias [22], and Lucid [23]. FIFO, SJF, QSSF, and Tiresias represent scheduling-only policies with different ordering assumptions, while Lucid is the strongest non-intrusive sharing baseline. All baselines use identical quota assignments and workload traces.

B. Job Execution Time Prediction Accuracy

We assessed the accuracy of our job execution time prediction algorithm in comparison to Lucid [23] using the Venus dataset. Table III summarizes the key metrics.

TABLE III: Accuracy metrics of job execution time prediction algorithms on the Venus dataset.

Algorithm	MAPE (%)	R^2 Score
Lucid	68.72	0.6413
DeepShare	31.84	0.7286

TABLE IV: Comparative GPU resource utilization across scheduling strategies.

Metric	FIFO	SJF	QSSF	Tiresias	Lucid	DeepShare
GPU Util (%)	39.64	40.00	39.40	39.27	54.52	70.58
Mem Util (%)	17.94	17.72	17.42	17.48	28.74	32.67

DeepShare reduces MAPE from 68.72% to 31.84% (a $2.16\times$ error reduction) and raises R^2 from 0.6413 to 0.7286 on the Venus dataset (Table III). Figure 4 confirms tight predicted-vs.-actual alignment, with accuracy highest for users with ≥ 50 historical submissions ($\text{MAPE} < 25$) and graceful degradation for cold-start users via the cluster-wide fallback ($\text{MAPE} < 60$).

C. Colocation Strategy Performance

We evaluated DeepShare’s interference-aware colocation strategy against non-sharing (FIFO, SJF, QSSF, Tiresias) and sharing (Lucid) baselines.

Resource utilization. Table IV compares GPU and memory utilization across all strategies.

DeepShare achieves 70.58% average GPU utilization and 32.67% memory utilization, representing a 29.5% relative improvement in GPU utilization over Lucid and a 78.1% improvement over FIFO. The utilization gain over Lucid is non-trivial because both systems perform colocation; the difference stems from three factors: (i) DeepShare’s more accurate interference model ($R^2 = 0.902$ vs. Lucid’s simpler scoring approach) enables it to accept more colocation pairs that Lucid conservatively rejects; (ii) the dynamic tolerance mechanism aggressively colocates jobs when contention is low (i.e., when $P = \left(\frac{G_p}{G_f + \epsilon}\right)^\gamma \approx 0$), exploiting off-peak periods that static thresholds cannot adapt to; and (iii) the DRA module channels additional Best-effort jobs into idle capacity, increasing the pool of colocation candidates.

The improvement in memory utilization over Lucid is more modest (13.7%) because GPU memory is a hard constraint that limits the scope of sharing. Figure 5 shows the temporal

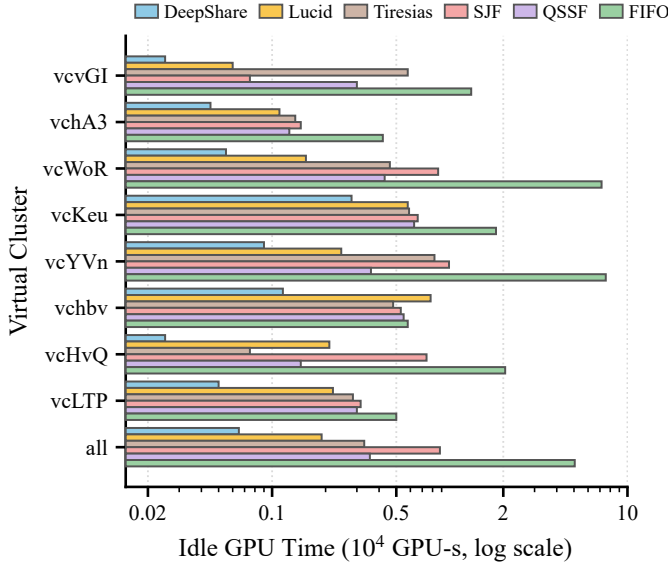


Fig. 6: Average queueing time across different Venus cluster configurations.

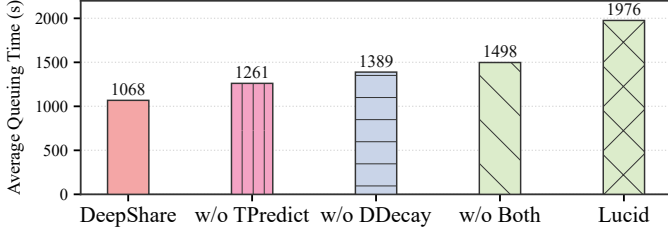


Fig. 7: The performance breakdown of queue waiting time.

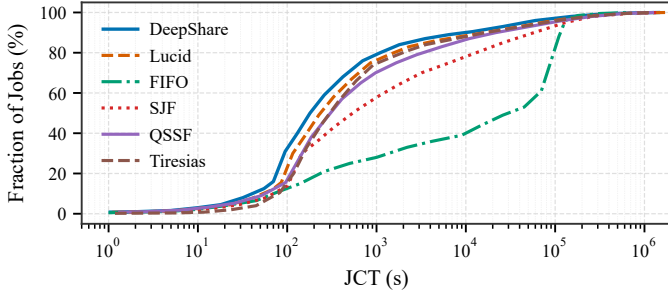


Fig. 8: CDF of job completion times.

dynamics of GPU utilization across strategies. DeepShare maintains a consistently higher and more stable utilization over time, with fewer periods of underutilization.

Job completion time and queueing delay. DeepShare achieves a 46% reduction in average queueing delay over Lucid (1,067 s vs. 1,976 s) and a 98% reduction over FIFO. The JCT improvement over Lucid is more modest (6.3%) because JCT includes both queueing and execution time, and execution time is inherently similar across scheduling policies for the same jobs. The dominant improvement is in queueing delay, which directly impacts user-perceived responsiveness.

TABLE V: Performance comparison of scheduling policies (all values in seconds).

Metric	FIFO	SJF	QSSF	Tiresias	Lucid	DeepShare
JCT	64,314	21,094	15,550	14,684	13,877	13,001
Wait	52,546	9,325	3,090	2,900	1,976	1,067

TABLE VI: Quota guarantee and queueing delay across strategies.

Strategy	Avg. $Q_i(t)$	Avg. Delay (s)
FIFO	1.00	98,000
SJF	1.00	42,387
QSSF	1.00	97,271
Tiresias	0.15	86,754
Real	0.75	4,466
DeepShare	1.00	1,168

Table V presents the JCT and queueing delay results.

The CDF of job completion times (Figure 8) shows that DeepShare achieves faster completion across all percentiles. Notably, the improvement is most pronounced at the tail (P95 and P99), where DeepShare reduces tail JCT by 23% compared to Lucid, indicating that it effectively prevents worst-case scenarios for long-queued jobs.

DeepShare consistently achieves the lowest idle GPU time across all tested Venus cluster configurations, shown in Figure 6. Averaged over the eight configurations, it reduces idle GPU time by 71.0% relative to Lucid and 96.8% relative to FIFO; in the aggregate all setting, idle GPU time further drops to 0.065×10^4 GPU-s, compared with 0.190×10^4 for Lucid and 5.05×10^4 for FIFO.

Ablation study. To understand the contribution of individual components, we conducted an ablation study as Figure 7 shows. The full DeepShare strategy reduces average queueing time by 45.9% relative to Lucid ($p < 0.01$, Wilcoxon signed-rank test). Removing execution-time prediction increases queueing time by 18.4%, while removing interference awareness increases it by 30.1%, showing that both components matter and that colocation contributes the larger share. These results should be interpreted together with the quota-management results in §V-D: runtime prediction and colocation reduce delay only when constrained by QAD, which prevents short-job optimization and aggressive sharing from overriding tenant recovery.

D. Multi-Tenant Quota Management

We evaluate DRA’s impact on fairness and queueing delay against fixed-quota schedulers, Tiresias, and real-world system traces (*Real*) from the internal cluster. We report average instantaneous QAD $Q_i(t)$ to measure raw quota satisfaction, and use smoothed $\bar{Q}_i(t)$ for cycle-level QoS compliance because it matches the scheduler’s control signal. Table VI presents the key metrics.

Among the evaluated strategies, DeepShare is the only one that achieves an average $Q_i(t)$ of 1.0 while maintaining low queueing delay (1,168 s). Fixed-quota strategies (FIFO, SJF,

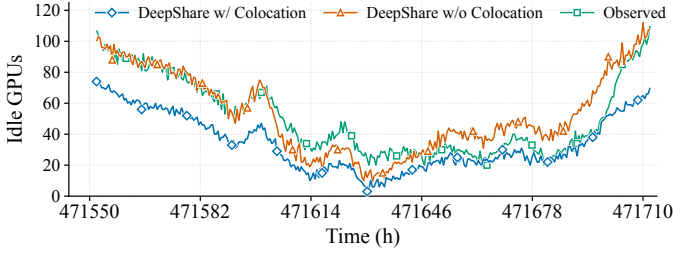


Fig. 9: Idle GPU counts over time.

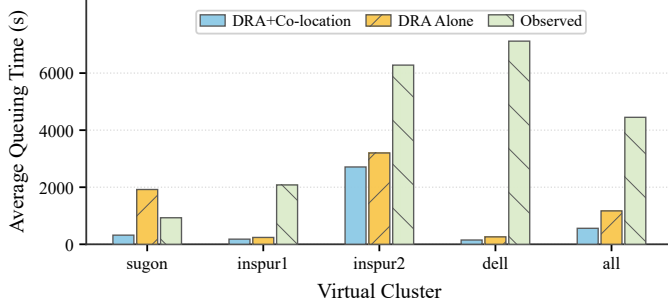


Fig. 10: Queueing delay with DRA alone vs. DRA + colocation. The additional 31% reduction confirms the benefit of coupled operation.

QSSF) trivially achieve average $\tilde{Q}_i(t)$ of 1.0 but at the cost of extremely high queueing delays ($36\text{--}84\times$ higher). Tiresias improves scheduling efficiency but severely violates quota guarantees (average $\tilde{Q}_i(t)$ of 0.15), making it unsuitable for multi-tenant environments with SLAs. The real system traces show a compromise (average $\tilde{Q}_i(t)$ of 0.75, delay 4,466 s) that neither fully satisfies quotas nor minimizes delay.

DRA’s *elastic quota mechanism* makes this possible: tenants opportunistically use idle capacity, while QAD keeps borrowed capacity reclaimable when Guaranteed demand becomes under-served. This decouples quota compliance from utilization, the two are no longer in tension.

Figure 9 shows the time-series of idle GPU counts. DeepShare maintains consistently fewer idle GPUs, confirming that its elastic allocation effectively fills resource gaps.

QoS compliance analysis. A scheduling cycle is compliant for tenant i if $\tilde{Q}_i(t) \geq 0.95$. In the physical deployment (§V-E), 93% of tenant-cycle pairs meet this threshold. The remaining 7% of violations concentrate during two scenarios: (i) burst arrivals where multiple tenants temporarily exceed aggregate capacity, and (ii) transient periods immediately following preemption events, before $\tilde{Q}_i(t)$ recovers. In the Venus simulation, the median per-tenant $\tilde{Q}_i(t)$ is 0.98 (IQR 0.94–1.00), and no tenant experiences $\tilde{Q}_i(t) < 0.85$ for more than 2% of scheduling cycles, confirming per-tenant (not just aggregate) compliance.

Overload behavior. When aggregate Guaranteed demand exceeds cluster capacity (approximately 8% of Venus peak-hour cycles), DeepShare degrades gracefully: Algorithm 1 prioritizes the most under-served tenants by ascending $\tilde{Q}_i(t)$

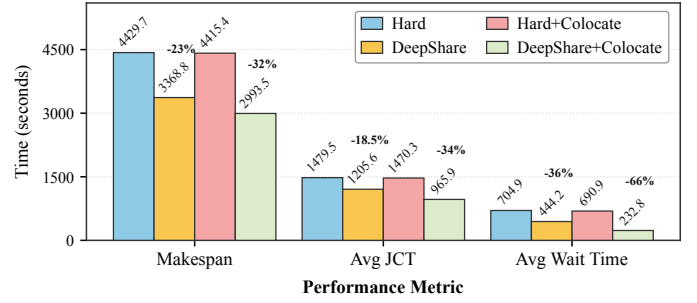


Fig. 11: Physical cluster results. DeepShare with colocation achieves the best performance across all metrics, confirming practical deployability.

order. The worst-case per-tenant $\tilde{Q}_i(t)$ during overload is 0.72, with recovery to $\tilde{Q}_i(t) \geq 0.95$ within 3.2 cycles (~ 160 ms) after the spike subsides. Best-effort jobs absorb most of the overload cost ($2.1\times$ queueing delay increase vs. only 14% for Guaranteed jobs), confirming the intended service differentiation.

Synergy with colocation. Figure 10 demonstrates that combining DRA with interference-aware colocation yields an additional 31% reduction in queueing delay compared to DRA alone. This synergy arises because colocation increases the cluster’s usable capacity by placing two jobs per GPU where possible, while QAD prevents those placements from delaying recovery of under-served tenants.

E. Evaluation on a Physical Cluster

To validate practical deployability, we evaluated DeepShare on our Kubernetes-managed testbed. We ran 50 jobs following internal cluster patterns (1-GPU: 77%, 2-GPU: 15%, 4-GPU: 8%) and measured JCT, queueing time, and makespan.

Figure 11 presents the results under four configurations: **Hard** (fixed quota, no colocation), **Hard+Colocate** (fixed quota with colocation), **DeepShare** (DRA without colocation), and **DeepShare+Colocate** (full system).

With DRA alone, makespan decreased from 4,429.7 s to 3,368.8 s (23% reduction), average JCT from 1,479.5 s to 1,205.6 s (18.5% reduction), and queueing delay from 704.9 s to 444.2 s (36% reduction). With the full system (DRA + Colocation), compared to Hard+Colocate, makespan reduced by 32%, average JCT by 34%, and queueing delay by 66% (from 690.9 s to 232.8 s). The full system processes the same 50-job batch in 67.6% of the time required by the Hard baseline, corresponding to approximately $1.48\times$ higher throughput on the physical testbed.

The physical cluster improvements are slightly lower than simulation results, which is expected due to the smaller scale (16 GPUs vs. simulated hundreds) limiting the opportunities for elastic resource redistribution. Nevertheless, the consistent direction and magnitude of improvements across both environments confirm the robustness of DeepShare’s design.

F. Sensitivity Analysis

We analyze the robustness of DeepShare to its two most influential parameters.

Dynamic tolerance baseline ($\rho_{\min}$). We varied $\rho_{\min}$ from 0.5 to 0.95 and measured GPU utilization and average per-job degradation of colocated jobs. At $\rho_{\min} = 0.5$ (aggressive), GPU utilization increases to 71.2% but average degradation reaches 18%, causing a net negative impact on JCT. At $\rho_{\min} = 0.7$ (default), the system achieves 70.58% utilization with <8% average degradation. At $\rho_{\min} = 0.9$ (conservative), utilization drops to 55.3% as colocation opportunities become too restricted. The default thus provides a robust balance between cluster efficiency and individual job impact.

Preemption cost weight (α). We varied α from 0 to 2.0. Higher values reduce repeated preemptions of the same job but may cause suboptimal resource reclamation. We find $\alpha \in [0.3, 0.8]$ consistently near-optimal across both datasets, with default $\alpha = 0.5$. The multi-victim penalty β exhibits similar robustness across $[0.1, 0.6]$.

G. Scalability and Limitations

The main scheduling components scale linearly in jobs and GPUs; the interference model has constant-time inference per candidate pair; multi-victim preemption search is bounded by the greedy heuristic (§III-B). Extending DeepShare to heterogeneous accelerators requires retraining the interference model on per-device profiling data.

Our interference predictor is trained on 16 DL model families (CV, NLP, RL, recommender); unseen architectures may exhibit different contention patterns, though the model’s reliance on hardware-level counters (SM activity, memory bandwidth) provides generalization across frameworks. The physical testbed (16 GPUs) is smaller than production clusters; results are most directly applicable to departmental-scale clusters (tens to low hundreds of GPUs), while simulation on 23,859 jobs provides evidence at larger scale.

VI. RELATED WORK

GPU cluster scheduling and workload characterization. Large-scale traces have revealed persistent under-utilization in multi-tenant GPU clusters [9], [10], [14]. Scheduling policies address this from different angles: Tiresias [22] prioritizes jobs via a multi-level feedback queue without runtime knowledge; Chronus [35] targets deadline-aware scheduling; Pollux [36] co-adapts batch sizes and allocations for goodput maximization. Other recent policies target notebook reclamation [37], serverless ML [38], non-linear multi-GPU scalability [39], and heterogeneous GPU placement [40]. None of these systems jointly optimize quota assurance, interference-aware colocation, and runtime-predictive preemption within a single coordinated loop.

Interference-aware GPU sharing and colocation. Gandiva [20] and AntMan [8] pioneer DL job colocation with framework-level and asymmetric memory-scaling mechanisms, respectively. TGS [41] achieves transparent sharing

via driver-level rate control, while Orion [15] partitions SMs and memory at the kernel level for inference. Other systems address GPU fraction management [42] and memory-layer multiplexing [43]. On the modeling side, SCHED-TUNE [44] profiles interference on heterogeneous GPUs, Jacquet et al. [45] characterize per-job GPU power under sharing, and ISACPP [32] advances interference prediction with graph attention networks at 50–200 ms latency per pair. DeepShare differs in two respects: (i) its Random Forest predictor achieves sub-millisecond inference, meeting real-time scheduling budgets, and (ii) admission is gated by a dynamic, dual-sided quota-aware tolerance (Eq. 5) rather than a static threshold.

Fairness, quota management, preemption, and runtime prediction. Themis [46], ASTRAEA [47], and Shockwave [19] formalize various fairness objectives but do not differentiate production from best-effort workloads or integrate colocation. Gavel [17] optimizes max-min fairness over effective throughput but assumes exclusive GPU placement. HiveD [13] guarantees resources via static cell partitioning; Sia [18] optimizes goodput across heterogeneous GPUs but leaves colocation and quota out of scope. Other work addresses priority backfilling for HPC [48] and MPI process malleability [49]. In the Kubernetes ecosystem, the ElasticQuota plugin [24] and Volcano [25] provide namespace-level min/max quotas with binary reclamation. For preemption, REEF [50] targets microsecond kernel-level preemption for inference, GPREEPT [51] generalizes GPU preemptive mechanisms, GFS [52] forecasts organizational demand using checkpoint recency, and Parcae [53] migrates LLM training proactively. On runtime prediction, Optimus [21] fits loss curves online and ElasticFlow [54] scales workers by predicted throughput. In contrast, DeepShare introduces a continuous Quota Assurance Degree $\bar{Q}_i(t) \in [0, 1]$ that simultaneously drives scheduling priority, colocation admission, and QoS reporting, and combines predicted remaining time with a cost function that integrates progress loss and restart overhead for preemption decisions—capabilities absent from any single prior system.

VII. CONCLUSION

This paper presented DeepShare, an assurance-driven resource management framework for multi-tenant GPU clusters that coordinates elastic quota regulation, predictive scheduling, and interference-aware colocation through the Quota Assurance Degree $\bar{Q}_i(t)$. Trace-driven simulation on 23,859 jobs shows 70.58% average GPU utilization (29.5% above Lucid) and 46% lower queueing delay than Lucid, while a 16-GPU Kubernetes deployment confirms a 34% JCT reduction and 93% per-tenant QoS compliance. Combining DRA with colocation further reduces queueing delay by 31% beyond DRA alone. Future work includes extending DeepShare to heterogeneous GPU environments and larger distributed training jobs.

REFERENCES

- [1] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [2] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, “Deepseek-v3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2015.
- [5] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu *et al.*, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [6] A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon, and E. D. Cubuk, “Scaling deep learning for materials discovery,” *Nature*, vol. 624, no. 7990, pp. 80–85, 2023.
- [7] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *Proceedings of the Tenth European Conference on Computer Systems*, ser. EuroSys ’15, 2015, pp. 1–17.
- [8] W. Xiao, S. Ren, Y. Li, Y. Zhang, P. Hou, Z. Li, Y. Feng, W. Lin, and Y. Jia, “AntMan: Dynamic scaling on GPU clusters for deep learning,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 533–548. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/xiao>
- [9] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, “Analysis of Large-Scale Multi-Tenant GPU clusters for DNN training workloads,” in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, 2019, pp. 947–960. [Online]. Available: <https://www.usenix.org/conference/atc19/presentation/jeon>
- [10] Q. Weng, W. Xiao, Y. Yu, W. Wang, C. Wang, J. He, Y. Li, L. Zhang, W. Lin, and Y. Ding, “MLaaS in the wild: Workload analysis and scheduling in Large-Scale heterogeneous GPU clusters,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, 2022, pp. 945–960. [Online]. Available: <https://www.usenix.org/conference/nsdi22/presentation/weng>
- [11] Q. Weng, L. Yang, Y. Yu, W. Wang, X. Tang, G. Yang, and L. Zhang, “Beware of fragmentation: Scheduling GPU-Sharing workloads with fragmentation gradient descent,” in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, 2023, pp. 995–1008. [Online]. Available: <https://www.usenix.org/conference/atc23/presentation/weng>
- [12] X. Sun, C. Hu, R. Yang, P. Garraghan, T. Wo, J. Xu, J. Zhu, and C. Li, “Rose: Cluster resource scheduling via speculative over-subscription,” in *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2018, pp. 949–960.
- [13] H. Zhao, Z. Han, Z. Yang, Q. Zhang, F. Yang, L. Zhou, M. Yang, F. C. Lau, Y. Wang, Y. Xiong, and B. Wang, “HiveD: Sharing a GPU cluster for deep learning with guarantees,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 515–532. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/zhao-hanyu>
- [14] Q. Hu, Z. Ye, Z. Wang, G. Wang, M. Zhang, Q. Chen, P. Sun, D. Lin, X. Wang, Y. Luo, Y. Wen, and T. Zhang, “Characterization of large language model development in the datacenter,” in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 709–729. [Online]. Available: <https://www.usenix.org/conference/nsdi24/presentation/hu>
- [15] F. Strati, X. Ma, and A. Klimovic, “Orion: Interference-aware, fine-grained GPU sharing for ML applications,” in *Proceedings of the Nineteenth European Conference on Computer Systems*, ser. EuroSys ’24, 2024, pp. 1075–1092.
- [16] R. Gu, Y. Chen, S. Liu, H. Dai, G. Chen, K. Zhang, Y. Che, and Y. Huang, “Liquid: Intelligent resource estimation and network-efficient scheduling for deep learning jobs on distributed gpu clusters,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2808–2820, 2022.
- [17] D. Narayanan, K. Santhanam, F. Kazhemiaka, A. Phanishayee, and M. Zaharia, “Heterogeneity-Aware cluster scheduling policies for deep learning workloads,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 481–498. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/narayanan-deepak>
- [18] S. Jayaram Subramanya, D. Arfeen, S. Lin, A. Qiao, Z. Jia, and G. R. Ganger, “Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling,” in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP ’23, 2023, pp. 642–657.
- [19] P. Zheng, R. Pan, T. Khan, S. Venkataraman, and A. Akella, “Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 703–723. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/zheng>
- [20] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, P. Patel, X. Peng, H. Zhao, Q. Zhang, F. Yang, and L. Zhou, “Gandiva: Introspective cluster scheduling for deep learning,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, 2018, pp. 595–610. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/xiao>
- [21] Y. Peng, Y. Bao, Y. Chen, C. Wu, and C. Guo, “Optimus: An efficient dynamic resource scheduler for deep learning clusters,” in *Proceedings of the Thirteenth EuroSys Conference*, ser. EuroSys ’18, 2018, pp. 1–14.
- [22] J. Gu, M. Chowdhury, K. G. Shin, Y. Zhu, M. Jeon, J. Qian, H. Liu, and C. Guo, “Tiresias: A GPU cluster manager for distributed deep learning,” in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, 2019, pp. 485–500. [Online]. Available: <https://www.usenix.org/conference/nsdi19/presentation/gu>
- [23] Q. Hu, M. Zhang, P. Sun, Y. Wen, and T. Zhang, “Lucid: A non-intrusive, scalable and interpretable scheduler for deep learning training jobs,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 457–472.
- [24] Kubernetes SIG Scheduling, “Kubernetes scheduler plugins: ElasticQuota,” <https://github.com/kubernetes-sigs/scheduler-plugins/tree/master/pkg/capacityscheduling>, 2024, accessed: 2025-03-01.
- [25] Volcano Community, “Volcano: Cloud native batch system for high-performance workloads,” <https://volcano.sh/>, 2024, accessed: 2025-03-01.
- [26] Q. Hu, P. Sun, S. Yan, Y. Wen, and T. Zhang, “Characterization and prediction of deep learning workloads in large-scale GPU datacenters,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’21, 2021, pp. 1–15.
- [27] Z. Chen, W. Quan, M. Wen, J. Fang, J. Yu, C. Zhang, and L. Luo, “Deep learning research and development platform: Characterizing and scheduling with QoS guarantees on GPU clusters,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 1, pp. 34–50, 2020.
- [28] Z. Ye, W. Gao, Q. Hu, P. Sun, X. Wang, Y. Luo, T. Zhang, and Y. Wen, “Deep learning workload scheduling in GPU datacenters: A survey,” *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–38, 2024.
- [29] NVIDIA Corporation, *GPU Operator with MIG - NVIDIA Documentation Hub*, 2024. [Online]. Available: <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-operator-mig.html>
- [30] —, *Multi-Process Service - NVIDIA Documentation Hub*, 2024. [Online]. Available: <https://docs.nvidia.com/deploy/mps/index.html>
- [31] J. Liu, Z. Cai, Y. Liu, H. Li, Z. Zhang, R. Ma, and R. Buyya, “Smore: Enhancing gpu utilization in deep learning clusters by serverless-based co-location scheduling,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 5, pp. 903–917, 2025.
- [32] Z. Liu, Y. Cheng, C. Chen, J. Hu, R. Fu, and D. Zhang, “ISACPP: Interference-aware scheduling approach for deep learning training workloads based on co-location performance prediction,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 8, pp. 1591–1607, 2025.
- [33] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, B. Saha, C. Curino, O. O’Malley, S. Radia, B. Reed, and E. Baldeschwieler, “Apache Hadoop YARN: Yet another resource negotiator,” in *Proceedings of the 4th Annual Symposium on Cloud Computing*, ser. SoCC ’13. Association for Computing Machinery, 2013, pp. 1–16.

- [34] V. Svetnik, A. Liaw, C. Tong, J. C. Culberson, R. P. Sheridan, and B. P. Feuston, "Random forest: a classification and regression tool for compound classification and qsar modeling," *Journal of chemical information and computer sciences*, vol. 43, no. 6, pp. 1947–1958, 2003.
- [35] W. Gao, Z. Ye, P. Sun, Y. Wen, and T. Zhang, "Chronus: A novel deadline-aware scheduler for deep learning training jobs," in *Proceedings of the ACM Symposium on Cloud Computing*, 2021, pp. 609–623.
- [36] A. Qiao, S. K. Choe, S. J. Subramanya, W. Neiswanger, Q. Ho, H. Zhang, G. R. Ganger, and E. P. Xing, "Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 2021, pp. 1–18. [Online]. Available: <https://www.usenix.org/conference/osdi21/presentation/qiao>
- [37] B. Carver, J. Zhang, H. Wang, K. Mahadik, and Y. Cheng, "NotebookOS: A replicated notebook platform for interactive training with on-demand GPUs," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '26. Association for Computing Machinery, 2026, pp. 183–202.
- [38] H. Wu, J. Deng, H. Fan, S. Ibrahim, S. Wu, and H. Jin, "QoS-Aware and cost-efficient dynamic resource allocation for serverless ML workflows," in *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2023, pp. 886–896.
- [39] J. Han, M. M. Rafique, L. Xu, A. R. Butt, S.-H. Lim, and S. S. Vazhkudai, "MARBLE: A multi-GPU aware job scheduler for deep learning on HPC systems," in *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. IEEE, 2020, pp. 272–281.
- [40] S. Dongare, R. I. S. Khan, H. Albahar, N. Zhao, D. Meléndez-Maita, and A. R. Butt, "Hybrid learning and optimization-based dynamic scheduling for DL workloads on heterogeneous GPU clusters," in *Proceedings of the 2025 ACM Symposium on Cloud Computing (SoCC)*, ser. SoCC '25. Association for Computing Machinery, 2026, pp. 557–570.
- [41] B. Wu, Z. Zhang, Z. Bai, X. Liu, and X. Jin, "Transparent GPU sharing in container clouds for deep learning workloads," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 69–85. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/wu>
- [42] T.-A. Yeh, H.-H. Chen, and J. Chou, "KubeShare: A framework to manage GPUs as first-class and shared resources in container cloud," in *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '20, 2020, pp. 173–184.
- [43] P. Yu and M. Chowdhury, "Salus: Fine-grained GPU sharing primitives for deep learning applications," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 98–111.
- [44] H. Albahar, S. Dongare, Y. Du, N. Zhao, A. K. Paul, and A. R. Butt, "SCHEDTUNE: A heterogeneity-aware GPU scheduler for deep learning," in *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2022, pp. 695–705.
- [45] P. Jacquet, M. Agusti, E. Caron, C. Coti, M. D. De Assunção, L. Lefèvre, and A.-C. Orgerie, "Untangling GPU power consumption: Job-level inference in cloud shared settings," in *Proceedings of the 21st European Conference on Computer Systems*. Association for Computing Machinery, 2026, pp. 624–640.
- [46] K. Mahajan, A. Balasubramanian, A. Singhvi, S. Venkataraman, A. Akella, A. Phanishayee, and S. Chawla, "Themis: Fair and efficient GPU cluster scheduling," in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, 2020, pp. 289–304. [Online]. Available: <https://www.usenix.org/conference/nsdi20/presentation/mahajan>
- [47] Z. Ye, P. Sun, W. Gao, T. Zhang, X. Wang, S. Yan, and Y. Luo, "Astraea: A fair deep learning scheduler for multi-tenant gpu clusters," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2781–2793, 2022.
- [48] A. Gainaru, S. Klasky, and G. Pallez, "Priority-BF: a task manager for priority-based scheduling," in *Euro-Par 2025: Parallel Processing*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland, 2025, pp. 219–232.
- [49] S. Iserte, I. Martín-Álvarez, K. Rojek, J. I. Aliaga, M. Castillo, W. Folwarska, and A. J. Peña, "Resource optimization with MPI process malleability for dynamic workloads in HPC clusters," *Future Generation Computer Systems*, vol. 174, p. 107949, 2026.
- [50] M. Han, H. Zhang, R. Chen, and H. Chen, "Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 2022, pp. 539–558. [Online]. Available: <https://www.usenix.org/conference/osdi22/presentation/han>
- [51] R. Fan, T. Ren, M. Xie, S. Gao, J. Shu, and Y. Lu, "GPREEMPT: GPU preemptive scheduling made general and efficient," in *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, 2025, pp. 263–272. [Online]. Available: <https://www.usenix.org/conference/atc25/presentation/fan>
- [52] J. Duan, S. Xu, S. Qian, D. Yang, K. Wang, C. Liao, Y. Yu, Q. Hua, H. Hu, Q. Wang, W. Wu, D. Bao, T. Lu, J. Cao, G. Xue, G. Yang, L. Zhang, and G. Chen, "GFS: A preemption-aware scheduling framework for GPU clusters with predictive spot instance management," in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ACM, 2026, pp. 117–131.
- [53] J. Duan, Z. Song, X. Miao, X. Xi, D. Lin, H. Xu, M. Zhang, and Z. Jia, "Parcae: Proactive, Liveput-Optimized DNN training on preemptible instances," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 2024, pp. 1121–1139. [Online]. Available: <https://www.usenix.org/conference/nsdi24/presentation/duan>
- [54] D. Gu, Y. Zhao, Y. Zhong, Y. Xiong, Z. Han, P. Cheng, F. Yang, G. Huang, X. Jin, and X. Liu, "ElasticFlow: An elastic serverless training platform for distributed deep learning," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 266–280.